matplotc: uma biblioteca de plotagem de gráficos em C

Felipe Cassio Rocha de Oliveira Lima[1], **João Augusto Haupt Fonseca Oliveira**[1], **Mateus Cruzatto Ramos**[1], **Sarah Antunes**[1]

[1]Instituto de Matemática, Estatística e Ciência da Computação da Universidade de São Paulo (IME-USP) – Caixa Postal 05508-090 – São Paulo, SP – Brasil

{felipecassio, joao.haupt, mateus.cruzatto, sarahantunesvilela}@usp.br

Abstract. *Matplotc is a modular charting library written in C11 on top of the raylib graphics library. Inspired by Python's matplotlib, it exposes a small public API, split into four modules (context, data, charts and export), that lets a client program build line, scatter, bar, histogram and pie charts, either showing them in an interactive window or exporting them to PNG files. The project is built with a Makefile that produces both a static (`.a`) and a shared (`.so`) library, has automated tests run by `make test`, and was developed incrementally with Git, using feature branches and pull requests.*

Resumo. *O matplotc é uma biblioteca modular de plotagem de gráficos escrita em C11 sobre a biblioteca gráfica raylib. Inspirada no matplotlib do Python, ela expõe uma API pública enxuta, dividida em quatro módulos (contexto, dados, gráficos e exportação), que permite a um programa cliente construir gráficos de linha, dispersão, barras, histograma e pizza, exibindo-os em uma janela interativa ou exportando-os para arquivos PNG. O projeto é compilado por um Makefile que gera uma biblioteca estática (`.a`) e uma dinâmica (`.so`), possui testes automatizados executados por `make test` e foi desenvolvido de forma incremental com Git, utilizando branches de funcionalidade e pull requests.*

1. Visão geral do projeto

A linguagem C não dispõe, em sua biblioteca padrão, de qualquer recurso para visualização de dados. Quem precisa desenhar um gráfico simples a partir de um programa em C costuma exportar os números para outra ferramenta (uma planilha, um *script* em Python) ou lidar diretamente com uma API gráfica de baixo nível, calculando à mão escalas, eixos, marcações e legendas.

O domínio escolhido para este trabalho é, portanto, o de plotagem de gráficos. O matplotc é uma biblioteca reutilizável que encapsula esse trabalho: o usuário informa apenas os dados, o intervalo dos eixos e o tipo de gráfico desejado, e a biblioteca cuida do plano cartesiano, das escalas, das marcações, dos rótulos e da legenda. O nome e o modelo de uso são inspirados no `matplotlib`, a biblioteca de plotagem mais difundida do Python: primeiro se *monta* o gráfico e depois ele é *exibido* na tela ou *exportado* para um arquivo.

1.1. Escopo

A versão atual do matplotc oferece:

- cinco tipos de gráfico: linhas, dispersão (com reta de regressão linear opcional), barras, histograma e pizza;

- suporte a várias séries de dados no mesmo gráfico, cada uma com sua cor e sua entrada na legenda;
- plano cartesiano configurável (intervalos, passo das marcações e linhas de grade opcionais);
- criação de dados a partir de vetores em memória ou carregamento a partir de arquivos texto (`.txt`/`.csv`) de duas colunas;
- exibição em janela interativa, com captura de tela pela tecla `S`;
- exportação direta para PNG, sem abrir janela visível.

Ficam fora do escopo, nesta versão, gráficos 3D, interação com o mouse (zoom, *pan*), múltiplos gráficos por janela (*subplots*) e ajuste automático dos eixos aos dados.

1.2. Tecnologias utilizadas

- **C11** (`-std=c11`), compilado com **GCC** e as opções `-Wall -Wextra -O2`;
- **raylib 5.5**, biblioteca gráfica escrita em C, usada para abrir a janela, desenhar as primitivas (linhas, círculos, retângulos, setores circulares e texto) e exportar imagens. Ela foi escolhida por ter uma API simples, sem dependências pesadas e com funções de desenho de alto nível suficientes para o projeto;
- **GNU Make**, para a compilação incremental, geração das bibliotecas, testes e instalação;
- **Git** e **GitHub**, para o controle de versão e a integração das contribuições por *pull requests*;
- **Linux** como ambiente de execução (o *script* de dependências também prevê o MSYS2 no Windows).

O repositório completo do projeto pode ser encontrado aqui.

2. Requisitos e dependências

A única dependência externa é a raylib (cabeçalho e biblioteca), que no Linux exige ainda os cabeçalhos de desenvolvimento de X11 e OpenGL. Para facilitar a preparação do ambiente, o projeto traz o *script* `scripts/setup-deps.sh`, acionado por `make deps`. Ele detecta a distribuição, tenta instalar a raylib pelo gerenciador de pacotes e, se ela não estiver disponível, compila a versão fixada (5.5.0) em `~/.local`. O alvo `make check-deps` apenas verifica se o cabeçalho `raylib.h` pode ser encontrado.

```
make deps                              # instala/compila a raylib
make check-deps                        # verifica se a raylib esta
   disponivel
make RAYLIB_PREFIX=$HOME/.local        # raylib instalada fora do
   sistema
```

O Makefile localiza a raylib por meio do `pkg-config` e, caso ele não a encontre, recorre aos nomes padrão das bibliotecas de sistema (`-lraylib -lGL -lpthread -ldl -lrt -lX11`).

3. Arquitetura do software

3.1. Estrutura do repositório

O projeto segue a estrutura sugerida no enunciado, com uma separação clara entre biblioteca, interface pública, interface interna, testes e aplicação cliente:

```
matplotc/
|-- Makefile
|-- app/
|   `-- main.c              # aplicacao cliente
|-- include/                # API publica (instalada)
|   |-- matplotc.h
|   |-- context.h
|   |-- data.h
|   |-- plane.h
|   `-- internal/           # cabecalhos privados (nao
   instalados)
|       |-- bar.h  hist.h  line.h  pie.h  scatter.h
|       |-- data_draw.h  plane_draw.h
|       `-- plotarea.h  colours.h
|-- src/                    # implementacao da biblioteca
|   |-- matplotc.c  context.c  plane.c  data.c  load.c
|   |-- line.c  scatter.c  bar.c  hist.c  pie.c
|   `-- plotarea.c  colours.c
|-- tests/
|   |-- framework.h  test_main.c  test_data.c  test_export.c
|   `-- fixtures/           # arquivos de dados para os testes
|-- examples/vendas.txt
|-- scripts/setup-deps.sh
|-- docs/README.md
|-- build/  lib/  bin/      # gerados pela compilacao
```

O código-fonte da biblioteca soma cerca de mil linhas em 12 arquivos `.c`, acompanhados de 4 cabeçalhos públicos e 9 cabeçalhos internos.

3.2. Camadas

A biblioteca é organizada em camadas, de modo que cada uma conhece apenas a camada imediatamente abaixo:

1. **API pública** (`include/*.h`): tipos `Context`, `Plane` e `Data` e as funções que o usuário chama;
2. **despachante** (`src/matplotc.c`): guarda o gráfico registrado e, a cada quadro, encaminha o desenho ao módulo correspondente;
3. **módulos de gráfico** (`line.c`, `scatter.c`, `bar.c`, `hist.c`, `pie.c`): compõem um quadro completo de cada tipo de gráfico;
4. **módulos de desenho e apoio** (`plane.c`, `data.c`, `plotarea.c`, `colours.c`): desenham eixos, rótulos e legenda, calculam a área útil e definem a paleta;
5. **raylib**: janela, contexto OpenGL e primitivas gráficas.

Somente as camadas 3 e 4 incluem `raylib.h`. Os cabeçalhos públicos não dependem da raylib, de modo que a aplicação cliente não precisa conhecer a biblioteca gráfica utilizada.

3.3. Fluxo de uso

O fluxo típico de uso da biblioteca, também descrito no README, é:

```
create_context  -.
create_plane     |  monta os objetos (sem abrir janela)
create_data     -+
plot_bar         |  registra o grafico escolhido
                 +--> show()        -> janela interativa (loop
                    ate fechar)
                 `--> export_png()  -> arquivo PNG (sem janela
                    visivel)
free_data           libera os dados, se vieram de load_data
```

4. Decisões de arquitetura e projeto

Esta seção discute as principais escolhas de projeto, os motivos que as justificam e os seus custos.

4.1. Separar o registro do gráfico da sua renderização

A decisão central do matplotc é a separação entre *registrar* um gráfico e *renderizá-lo*. As funções `plot_*` não abrem janela nem desenham: elas apenas validam os argumentos e guardam, em uma estrutura estática interna, o tipo do gráfico e os ponteiros recebidos. Quem efetivamente desenha é `show()` (em uma janela) ou `export_png()` (em um arquivo).

```
1  typedef enum {
2    PLOT_LINE, PLOT_SCATTER, PLOT_BAR, PLOT_PIE, PLOT_HIST,
3  } PlotType;
4
5  typedef struct {
6    PlotType type;
7    Plane *plane;
8    Data *data;
9    int data_count;
10   bool corr;
11 } PendingPlot;
12
13 static PendingPlot pending = {0};
14 static bool has_pending = false;
```

Essa decisão surgiu da refatoração registrada no *commit* `cc53234` (*"separate plotting logic from drawing"*). Antes dela, cada `plot_*` abria a própria janela e continha o próprio laço de renderização, o que tornava impossível reutilizar o mesmo desenho para exportação e também impossibilitava testar as funções sem abrir uma janela. Com o registro separado:

- o mesmo código de desenho (`draw_frame`) serve à janela e à exportação PNG;
- as funções `plot_*` não bloqueiam e podem ser testadas em ambientes sem tela;
- o modelo de uso fica idêntico ao do `matplotlib` (`plt.plot(...)` seguido de `plt.show()` ou `plt.savefig(...)`).

O custo é a existência de um **estado global**: só há um gráfico registrado por vez (uma nova chamada a `plot_*` substitui a anterior), a biblioteca não é segura para uso concorrente por várias *threads*, e os ponteiros passados a `plot_*` precisam continuar válidos até a chamada de `show()` ou `export_png()`.

4.2. Interface pública separada da interface interna

Na refatoração do *commit* `fa91b46`, os cabeçalhos foram divididos em dois grupos. Os de `include/` formam a API pública e são os únicos instalados por `make install`; os de `include/internal/` declaram funções de uso exclusivo da própria biblioteca (por exemplo, `draw_plane`, `draw_legend` e `create_bar`). O Makefile acrescenta `-Iinclude/internal` somente na compilação da biblioteca, e o comentário no próprio Makefile deixa a intenção explícita:

```
# Cabecalhos publicos (o que o usuario inclui). Os internos em
# include/internal/ nao sao instalados: nao sao API.
HEADERS   := $(wildcard $(INC_DIR)/*.h)
```

Com isso, a aplicação cliente só enxerga `Context`, `Plane`, `Data` e as funções públicas. Tipos como `PlotArea` e `PendingPlot`, a paleta de cores e todas as rotinas de desenho podem mudar sem que a aplicação precise ser alterada, atendendo ao requisito 3.2 do enunciado. Funções que nem precisam aparecer em cabeçalho interno, como `append_point` em `load.c` e `draw_frame` em `matplotc.c`, são declaradas `static`.

4.3. Estruturas de dados passadas por valor

Os três tipos públicos são estruturas pequenas e transparentes, criadas por construtores que as devolvem **por valor**:

- `Context`: largura, altura e título da janela;
- `Plane`: intervalo e passo de cada eixo e a opção de linhas de grade;
- `Data`: uma série de dados, com os vetores `x` e `y`, o tamanho, a legenda da série e os rótulos dos eixos.

Retornar por valor dispensa alocação dinâmica para os objetos de configuração e, portanto, dispensa funções de destruição para eles: um `Context` ou um `Plane` vive na pilha do chamador como qualquer variável local. Várias séries são representadas simplesmente por um vetor de `Data` acompanhado do parâmetro `data_count`, o que mantém as assinaturas uniformes entre os tipos de gráfico.

Os campos são visíveis ao cliente (não são tipos opacos). Essa foi uma escolha consciente de simplicidade: os testes podem conferir diretamente os campos devolvidos pelos construtores, e o usuário pode inspecioná-los. A contrapartida é que mudar o leiaute dessas três estruturas é uma mudança de API e exige recompilar a aplicação.

4.4. Gerenciamento de memória: empréstimo e posse

O matplotc trata a memória dos dados com uma regra explícita de **posse**, registrada no campo `owns_memory` de `Data`:

- **empréstimo** (`create_data`): a biblioteca apenas guarda os ponteiros para os vetores do chamador, sem copiá-los. Não há alocação, e o chamador continua responsável pela memória (que normalmente é um vetor local na pilha). Nesse caso, `owns_memory` é `false`;

- **posse** (`load_data`): os vetores são alocados pela própria biblioteca durante a leitura do arquivo, e `owns_memory` é `true`. O usuário deve chamar `free_data`.

A função `free_data` consulta essa marca e só libera a memória que a própria biblioteca alocou. Ela é segura para `NULL`, para dados emprestados (nesse caso não faz nada) e é idempotente, pois zera os ponteiros e a marca após liberar:

```
void free_data(Data *data) {
  if (data == NULL) {
    return;
  }
  if (!data->owns_memory) {   /* arrays pertencem ao chamador */
    return;
  }
  free(data->x);
  free(data->y);
  data->x = NULL;
  data->y = NULL;
  data->size = 0;
  data->owns_memory = false;
}
```

Durante o carregamento, os vetores crescem por **duplicação da capacidade** (16, 32, 64, . . .), o que dá custo amortizado constante por ponto inserido e evita percorrer o arquivo duas vezes para contar as linhas. Em caso de falha de `realloc`, os blocos já alocados são preservados e liberados pelo chamador, de forma que não há vazamento nem nos caminhos de erro:

```
static bool append_point(float **x, float **y, int *count,
                         int *capacity, float px, float py) {
  if (*count == *capacity) {
    int new_capacity = *capacity == 0 ? 16 : *capacity * 2;

    float *new_x = realloc(*x, (size_t)new_capacity * sizeof(
       float));
    if (new_x == NULL) return false;
    *x = new_x;

    float *new_y = realloc(*y, (size_t)new_capacity * sizeof(
       float));
    if (new_y == NULL) return false;
    *y = new_y;

    *capacity = new_capacity;
  }
  (*x)[*count] = px;
  (*y)[*count] = py;
  (*count)++;
  return true;
}
```

Os rótulos (`label`, `x_axis_label`, `y_axis_label` e o título do contexto) também são emprestados: a biblioteca guarda os ponteiros, e o caso de uso esperado são literais de *string*.

4.5. Tratamento de erros consistente

Todas as funções públicas seguem a mesma convenção, documentada nos cabeçalhos:

- **construtores** (`create_context`, `create_plane`, `create_data` e `load_data`) validam os parâmetros e, em caso de erro, imprimem uma mensagem em `stderr` e devolvem a estrutura **zerada**. O chamador detecta o erro testando um campo que nunca é zero em um objeto válido (`size == 0` para `Data`, `width == 0` para `Context`);
- **funções de registro** (`plot_*`) ignoram silenciosamente argumentos inválidos (ponteiros nulos ou `data_count <= 0`) e não registram nada;
- **funções de saída** (`show` e `export_png`) recusam-se a abrir a janela quando não há gráfico registrado; `export_png` devolve `false` em qualquer falha.

Usar a estrutura zerada como valor sentinela evita parâmetros de saída e códigos de erro separados, ao custo de exigir que o chamador verifique o resultado. As validações aplicadas estão resumidas na Tabela 1.

Tabela 1 – Condições de erro tratadas pela API

Função	Condição de erro e comportamento
`create_context`	largura ou altura ≤ 0, título nulo ou vazio: devolve `Context` zerado.
`create_plane`	mínimo $\geq$ máximo em algum eixo ou passo ≤ 0: devolve `Plane` zerado.
`create_data`	vetores nulos, tamanho ≤ 0 ou rótulo nulo: devolve `Data` zerado.
`load_data`	parâmetro nulo, arquivo inexistente, nenhum ponto válido ou falta de memória: devolve `Data` zerado, sem vazar memória. Linhas malformadas e valores não finitos são ignorados.
`free_data`	ponteiro nulo ou dados emprestados: não faz nada.
`plot_*`	contexto, plano (exceto na pizza) ou dados nulos, ou `data_count` ≤ 0: não registra o gráfico.
`show`	contexto nulo ou nenhum gráfico registrado: retorna sem abrir janela.
`export_png`	contexto ou nome nulo, nome vazio, nenhum gráfico registrado ou falha na gravação: devolve `false`.

Fonte: Os autores.

4.6. Desacoplamento entre dados, plano e área de desenho

Três conceitos que costumam se misturar em código gráfico foram mantidos separados:

- o **plano** (`Plane`) descreve o sistema de coordenadas *dos dados* (por exemplo, de 0 a 100 com marcações de 10 em 10);
- a **área de desenho** (`PlotArea`, interna) descreve o retângulo *da tela*, em pixels, onde o plano será desenhado;
- os **dados** (`Data`) são apenas números e rótulos.

A conversão de coordenadas é feita por uma única fórmula, repetida nos módulos de gráfico, que mapeia linearmente o intervalo do plano no retângulo da área de desenho (o eixo y é invertido porque, na tela, ele cresce para baixo):

$$x_{tela} = left + \frac{x - x_{min}}{x_{max} - x_{min}} \cdot width, \qquad y_{tela} = bottom - \frac{y - y_{min}}{y_{max} - y_{min}} \cdot height.$$

A `PlotArea` é calculada a partir do tamanho da janela, com margens proporcionais (10% à esquerda, 5% à direita e no topo, 7,5% embaixo) limitadas a valores mínimos e máximos em pixels. Isso garante espaço para os rótulos dos eixos tanto em janelas pequenas quanto grandes. Como o plano é passado separadamente dos dados, o gráfico de pizza, que não tem eixos, simplesmente não recebe um plano (`plot_pie` não tem esse parâmetro).

Por fim, cada módulo de gráfico é composto a partir de rotinas de desenho comuns, o que evita duplicação e dá aparência uniforme a todos os gráficos:

```
1 void create_line(Plane *plane, Data *data, int data_count,
2                  PlotArea *plotarea) {
3   draw_plane(plane, plotarea);                    /* eixos e
       grade */
4   draw_line_data(data, data_count, plane, plotarea);
5   draw_label(data, plane, plotarea);              /* rotulos dos
        eixos */
6   draw_legend(data, data_count, plotarea);
7 }
```

4.7. Desempenho

A raylib trabalha em modo imediato: a cada quadro, a tela é limpa e tudo é desenhado novamente. Em `show()`, o laço é limitado a 60 quadros por segundo (`SetTargetFPS(60)`), e cada quadro custa $O(n)$ no número total de pontos para os gráficos de linha, dispersão, barras e pizza. Para os volumes de dados visados (dezenas a milhares de pontos), esse custo é desprezível, e o modo imediato elimina a necessidade de manter uma estrutura de cena ou invalidar regiões da tela.

Alguns cuidados específicos foram tomados:

- a `PlotArea` é calculada uma única vez antes do laço, e não a cada quadro;
- no histograma, cujo custo é $O(bins \cdot n)$, o número de classes é limitado à largura da área em pixels, já que classes mais estreitas que um pixel não seriam visíveis;
- a regressão linear é acumulada em `double` para reduzir erros de arredondamento nas somas de x^2 e xy;
- o gráfico de barras usa o modo *scissor* da raylib para recortar o desenho na área do gráfico, em vez de testar cada barra manualmente contra as bordas;

- `export_png` desenha um único quadro em uma janela oculta, sem entrar em laço.

Um ponto de melhoria é que, como o desenho é refeito a cada quadro, cálculos que não mudam entre quadros (como as frequências do histograma e os coeficientes da regressão) são recomputados 60 vezes por segundo. Para conjuntos de dados grandes, valeria guardá-los no momento do registro.

4.8. Uso de `float` nas coordenadas

As primeiras versões usavam `int` nos vetores de dados e nos limites dos eixos. Uma sequência de *commits* (`80f1860`, `bd74f2e`, `86136b5` e `3add3c1`) migrou as assinaturas e implementações para `float`, permitindo dados contínuos (medidas, médias, valores monetários) e passos fracionários nos eixos. O `float` também é o tipo usado nativamente pelas funções de desenho da raylib (`Vector2`, `Rectangle`), o que evita conversões.

4.9. Portabilidade da compilação

O Makefile usa `pkg-config` para descobrir os parâmetros da raylib, com um conjunto de bibliotecas padrão como reserva, e aceita a variável `RAYLIB_PREFIX` para instalações fora do sistema. O executável dinâmico é ligado com `-Wl,-rpath,'$ORIGIN/../lib'`, de modo que encontra a `libmatplotc.so` relativa à sua própria localização, sem exigir `LD_LIBRARY_PATH` nem instalação prévia.

5. Organização e responsabilidade dos módulos

A API pública é organizada em **quatro módulos funcionalmente distintos**, cada um correspondente a uma etapa do uso da biblioteca. A Tabela 2 relaciona os módulos com os arquivos que os implementam.

Tabela 2 – Módulos públicos e arquivos correspondentes

Módulo	**Arquivos**	**Responsabilidade**
Contexto	`context.h`, `context.c`, `plane.h`, `plane.c`, `show` em `matplotc.c`	Configuração da janela e do sistema de coordenadas; exibição interativa.
Dados	`data.h`, `data.c`, `load.c`	Criação, carregamento a partir de arquivo e liberação das séries de dados.
Gráficos	`matplotc.h`, `matplotc.c`, `line.c`, `scatter.c`, `bar.c`, `hist.c`, `pie.c`	Registro do gráfico e composição de cada tipo.
Exportação	`export_png` em `matplotc.c`	Renderização do gráfico registrado em arquivo PNG.

Fonte: Os autores.

A seguir, descreve-se cada arquivo do código-fonte.

5.1. `matplotc.h` e `matplotc.c`: ponto de entrada e despachante

`matplotc.h` é o cabeçalho *guarda-chuva*: inclui `context.h`, `data.h` e `plane.h` e declara `show`, as cinco funções `plot_*` e `export_png`. É o único cabeçalho que a aplicação cliente precisa incluir.

`matplotc.c` contém o estado do gráfico registrado (`PendingPlot`), as funções `plot_*`, que apenas validam e registram, e a função estática `draw_frame`, que encaminha o desenho ao módulo do tipo registrado por meio de um `switch`. `show()` inicializa a janela, calcula a área de desenho e executa o laço de renderização até o usuário fechar a janela; durante o laço, a tecla `S` grava uma captura de tela em `grafico.png`:

```
InitWindow(context->width, context->height, context->title);
SetTargetFPS(60);
PlotArea plot_area = create_plotarea(context);

while (!WindowShouldClose()) {
  BeginDrawing();
  ClearBackground(RAYWHITE);
  draw_frame(&plot_area);
  EndDrawing();

  if (IsKeyPressed(KEY_S)) {
    TakeScreenshot("grafico.png");
  }
}
CloseWindow();
```

`export_png()` reaproveita o mesmo `draw_frame`, mas cria a janela com a opção `FLAG_WINDOW_HIDDEN`, desenha um único quadro, lê o *framebuffer* com `LoadImageFromScreen` e o grava com `ExportImage`. A janela oculta é necessária porque a raylib precisa de um contexto OpenGL para desenhar, mesmo que nada seja mostrado.

5.2. `context.c`: contexto

Implementa `create_context`, que valida as dimensões e o título e devolve a estrutura `Context`. O contexto não guarda recursos da raylib: a janela só é criada dentro de `show` ou `export_png`, o que permite criar e testar contextos sem ambiente gráfico.

5.3. `plane.c`: plano cartesiano

Contém o construtor `create_plane` (público) e as rotinas internas de desenho do plano, declaradas em `internal/plane_draw.h`: `draw_plane`, que desenha os dois eixos e a borda da área, e `draw_axis_x`/`draw_axis_y`. Estas dividem o intervalo do eixo pelo passo, desenham uma marcação e um valor numérico (formatado com `%g`) em cada divisão e, se a opção `lines` estiver ativa, as linhas de grade intermediárias em cinza claro.

5.4. `data.c` e `load.c`: dados

`data.c` implementa `create_data` e `free_data`, já discutidas, além de rotinas internas comuns a todos os gráficos com eixos: `draw_label`, que escreve o rótulo

do eixo x centralizado abaixo da área e o do eixo y girado em $-90°$ à esquerda, e `draw_legend`, que desenha, no canto superior direito, um quadrado colorido e o nome de cada série.

`load.c` implementa `load_data`. A função lê o arquivo linha a linha com `fgets`, descarta tudo a partir de `#` (comentários), troca vírgulas por espaços para aceitar os dois separadores e extrai os dois valores com `sscanf`. Linhas vazias, malformadas ou com valores não finitos (`NaN`, infinito) são ignoradas. Um exemplo de arquivo aceito é:

```
# Vendas mensais
1, 45
2, 52
5 72      # separador por espaco tambem vale
```

5.5. `line.c`: gráfico de linhas

Converte cada ponto para coordenadas de tela e liga pontos consecutivos de cada série com segmentos de 3 pixels de espessura (`DrawLineEx`), na cor da série. Os pontos são ligados na ordem em que aparecem no vetor, sem ordenação.

5.6. `scatter.c`: gráfico de dispersão e regressão

Desenha cada ponto como um círculo de raio 5. Quando o parâmetro `corr` de `plot_scatter` é verdadeiro, a função `draw_linear_correlation` calcula, para cada série, a reta de mínimos quadrados $y = ax + b$:

$$a = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - \left(\sum x_i\right)^2}, \qquad b = \frac{\sum y_i - a\sum x_i}{n},$$

e a desenha de uma extremidade à outra do eixo x. Séries com menos de dois pontos ou com denominador praticamente nulo (todos os x iguais) são ignoradas para evitar divisão por zero.

5.7. `bar.c`: gráfico de barras

O módulo de barras resolve dois problemas: a largura das barras e o agrupamento de várias séries. A largura é derivada do menor espaçamento entre valores consecutivos de x (ou de um décimo do intervalo, se não houver espaçamento útil); cada posição ocupa 80% desse espaçamento, dividido igualmente entre as séries, que aparecem lado a lado. A base das barras é o valor zero quando ele está dentro do intervalo do eixo y, o que permite representar valores negativos com barras para baixo. O desenho é recortado à área do gráfico com `BeginScissorMode`.

5.8. `hist.c`: histograma

O histograma trata os valores de x como uma variável **contínua** (essa foi a correção do *commit* `d5e3122`). O intervalo do eixo x é dividido em classes de largura `scale_x`; para cada classe $[b_{min}, b_{max})$ conta-se quantas amostras caem nela (a última classe é fechada à direita, para incluir o máximo), e a frequência é desenhada como uma barra. O eixo y representa, portanto, a frequência, e o vetor `y` da série não é utilizado. Com várias séries, as barras de cada classe são divididas lado a lado.

5.9. `pie.c`: gráfico de pizza

Soma os valores de y de todas as séries e desenha cada valor como um setor circular (`DrawCircleSector`) de ângulo proporcional à sua participação no total. A legenda própria do gráfico de pizza tem uma entrada por fatia, e não por série, exibindo o valor de x como nome da categoria e o percentual correspondente. Esse módulo não usa `Plane`, eixos nem rótulos de eixo (correção do *commit* `b2b974b`, *"remove axis from pie"*).

5.10. Módulos internos de apoio

- **`plotarea.c`**: `create_plotarea` calcula as margens e o retângulo útil do gráfico a partir do `Context`;
- **`colours.c`**: define a paleta `COLOURS` com oito cores contrastantes (vermelho, azul, verde, laranja, roxo, bordô, azul-escuro e verde-escuro), usada na ordem das séries;
- **`internal/data_draw.h`**: declara as rotinas de desenho de dados, rótulos e legenda compartilhadas pelos módulos de gráfico.

6. Interface pública (API)

Para usar a biblioteca, a aplicação inclui apenas `matplotc.h` e liga com `libmatplotc.a` ou `libmatplotc.so`. A API é formada por **três tipos** e **doze funções públicas**, distribuídas pelos quatro módulos.

6.1. Tipos

```
typedef struct {
    float width;          /* largura da janela em pixels   */
    float height;         /* altura da janela em pixels    */
    const char *title;    /* titulo da janela              */
} Context;

typedef struct {
  float x_min_max[2];     /* {minimo, maximo} do eixo x    */
  float scale_x;          /* passo das marcacoes do eixo x */
  float y_min_max[2];     /* {minimo, maximo} do eixo y    */
  float scale_y;          /* passo das marcacoes do eixo y */
  bool lines;             /* desenha linhas de grade       */
} Plane;

typedef struct {
  Plane plane;
  char *label;            /* nome da serie (legenda)       */
  float *x;
  char *x_axis_label;
  float *y;
  char *y_axis_label;
  int size;               /* numero de pontos              */
  bool owns_memory;       /* true se alocado por load_data */
} Data;
```

6.2. Funções

A Tabela 3 resume as funções públicas. As condições de erro de cada uma foram descritas na Tabela 1.

Tabela 3 – Funções públicas do matplotc

Módulo	**Assinatura**	**Finalidade e retorno**
Contexto	`Context create_context(float width, float height, const char *title)`	Cria o contexto da janela. Devolve o `Context` ou um `Context` zerado.
Contexto	`Plane create_plane(float x_min, float x_max, float scale_x, float y_min, float y_max, float scale_y, bool lines)`	Cria o plano cartesiano com intervalos, passos e grade. Devolve o `Plane` ou um `Plane` zerado.
Contexto	`void show(Context *context)`	Abre a janela e exibe o gráfico registrado até que ela seja fechada. Bloqueante.
Dados	`Data create_data(Plane plane, const char *label, float *x, const char *x_axis_label, float *y, const char *y_axis_label, int size)`	Cria uma série que referencia os vetores do chamador (sem cópia).
Dados	`Data load_data(const char *path, Plane plane, const char *label, const char *x_axis_label, const char *y_axis_label)`	Lê uma série de um arquivo de duas colunas, alocando os vetores.
Dados	`void free_data(Data *data)`	Libera os vetores alocados por `load_data`.
Gráficos	`void plot_line(Context *context, Plane *plane, Data *data, int data_count)`	Registra um gráfico de linhas com `data_count` séries.
Gráficos	`void plot_scatter(Context *context, Plane *plane, Data *data, int data_count, bool corr)`	Registra um gráfico de dispersão; com `corr`, desenha a reta de regressão de cada série.
Gráficos	`void plot_bar(Context *context, Plane *plane, Data *data, int data_count)`	Registra um gráfico de barras agrupadas.

Módulo	Assinatura	Finalidade e retorno
Gráficos	`void plot_histogram(Context *context, Plane *plane, Data *data, int data_count)`	Registra um histograma dos valores de `x`, com classes de largura `scale_x`.
Gráficos	`void plot_pie(Context *context, Data *data, int data_count)`	Registra um gráfico de pizza com os valores de `y` (sem plano).
Exportação	`bool export_png(Context *context, const char *filename)`	Grava o gráfico registrado em PNG, sem janela visível. Devolve `true` em caso de sucesso.
Fonte: Os autores.		

6.3. Exemplos de uso

O roteiro mínimo para exibir um gráfico de linhas é:

```c
#include "matplotc.h"

int main(void) {
  Context ctx = create_context(1000, 800, "Meu grafico");
  Plane plane = create_plane(0, 10, 1.0f, 0, 100, 10.0f, true);
  float x[] = {1, 2, 3, 4, 5};
  float y[] = {10, 40, 35, 70, 90};
  Data d = create_data(plane, "Serie", x, "Tempo", y, "Valor",
      5);

  plot_line(&ctx, &plane, &d, 1);   /* registra o grafico */
  show(&ctx);                       /* janela interativa  */
  return 0;
}
```

Para comparar duas séries em um gráfico de dispersão com regressão, basta passar um vetor de `Data`:

```c
Data series[] = {
    create_data(plane, "Crescente",   x, "Tempo", y1, "Valor",
       10),
    create_data(plane, "Decrescente", x, "Tempo", y2, "Valor",
       10)};

plot_scatter(&ctx, &plane, series, 2, true);
show(&ctx);
```

E para carregar dados de um arquivo e gerar uma imagem sem abrir janela:

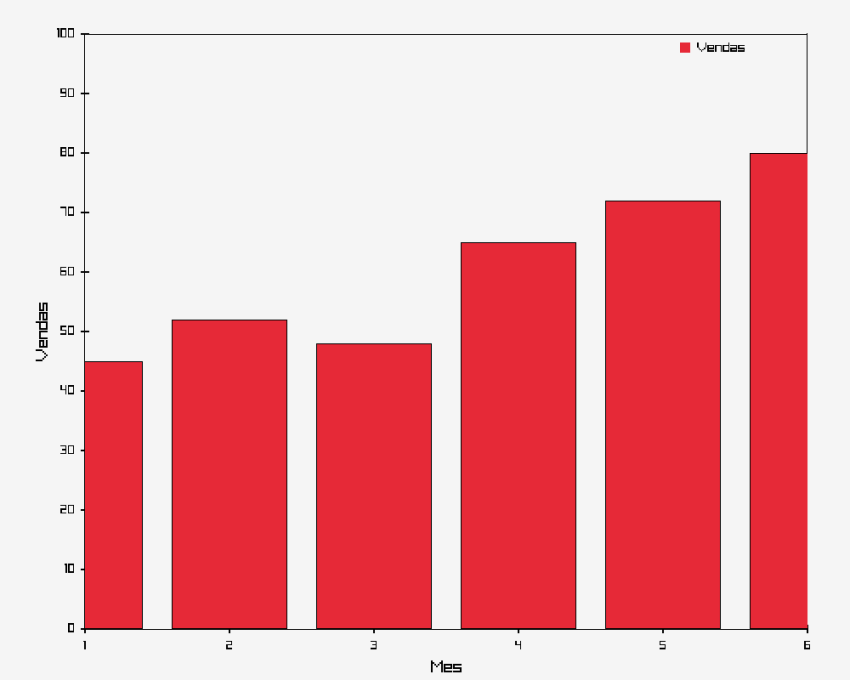
```c
Plane plane = create_plane(1, 6, 1.0f, 0, 100, 10.0f, false);
```

```
Data d = load_data("examples/vendas.txt", plane, "Vendas", "Mes"
    , "Vendas");
if (d.size == 0) {
  return 1;                          /* erro ja reportado em
     stderr */
}

plot_bar(&ctx, &plane, &d, 1);
if (export_png(&ctx, "grafico.png")) {
  printf("Imagem salva em grafico.png\n");
}
free_data(&d);                       /* memoria alocada por
   load_data */
```

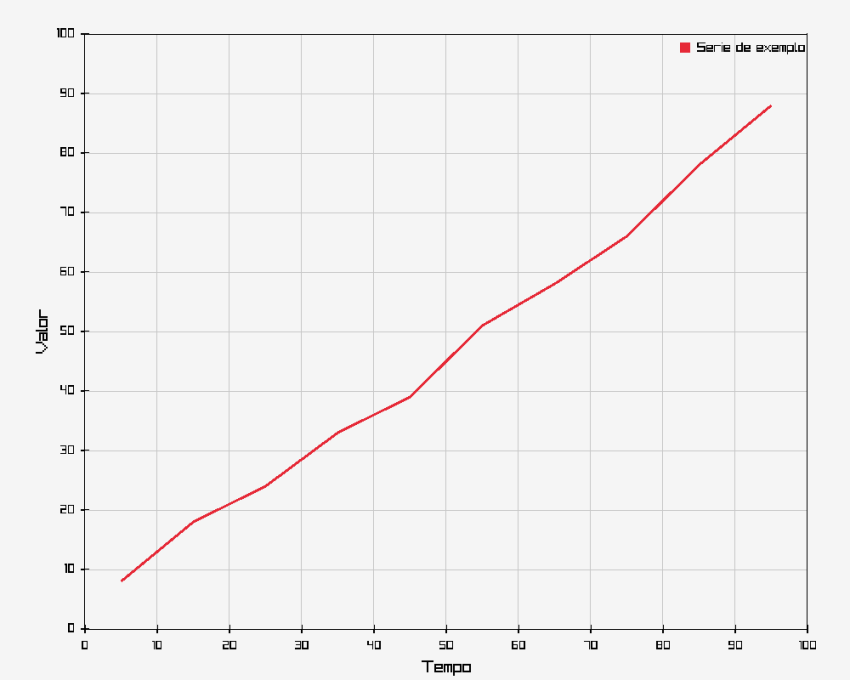
As Figuras 1 a 5 mostram os cinco tipos de gráfico produzidos pela biblioteca, exportados com `export_png` a partir dos mesmos dados da aplicação cliente.

Figura 1 – Gráfico de barras (`plot_bar`)



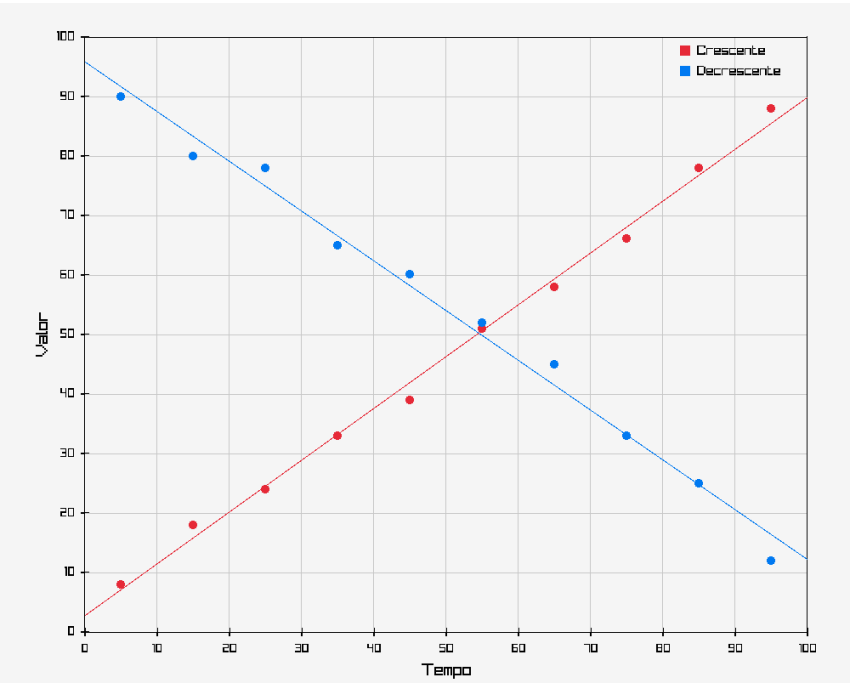
Fonte: Os autores.

Figura 2 – Gráfico de linhas com grade (`plot_line`)



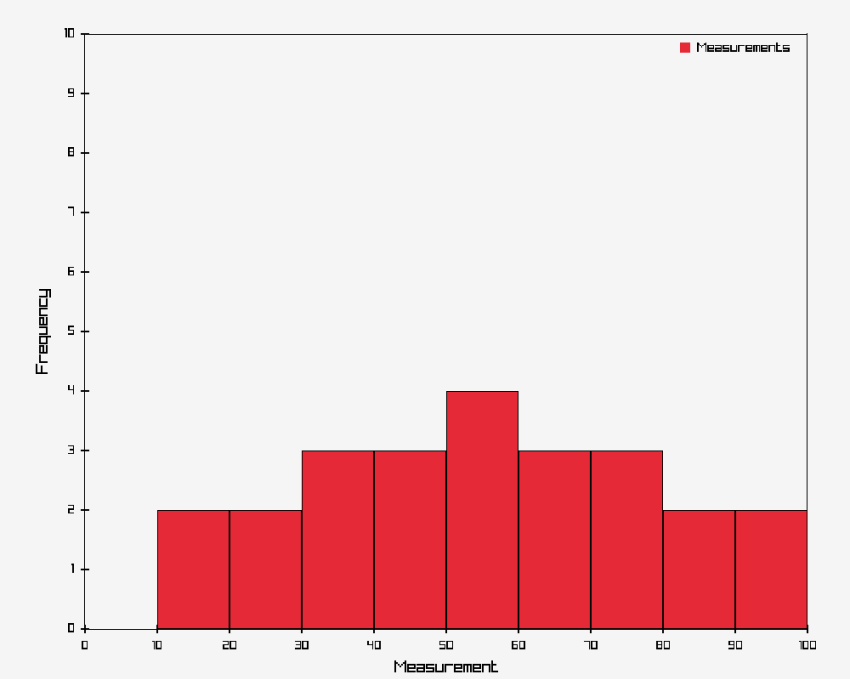
Fonte: Os autores.

Figura 3 – Dispersão de duas séries com retas de regressão (`plot_scatter`)



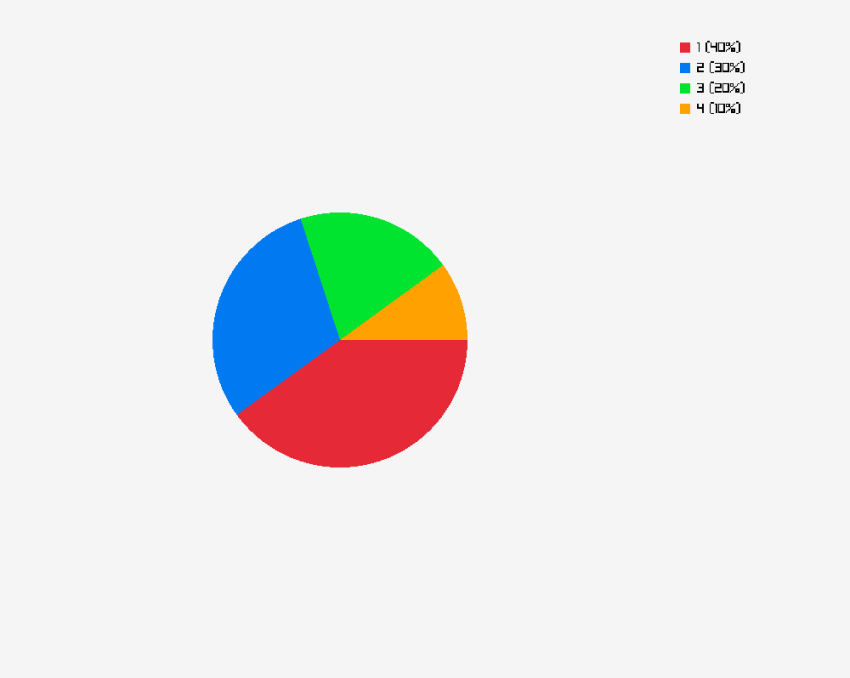
Fonte: Os autores.

Figura 4 – Histograma com classes de largura 10 (`plot_histogram`)



Fonte: Os autores.

Figura 5 – Gráfico de pizza com legenda percentual (`plot_pie`)



Fonte: Os autores.

7. Aplicação cliente

A aplicação cliente (`app/main.c`) é um programa independente que inclui apenas `matplotc.h` e nunca acessa arquivos `.c` ou cabeçalhos internos da biblioteca. Ela apresenta um menu com sete opções:

```
MatplotC - selecione um tipo de grafico:
1. Barras
2. Linha
3. Dispersao
4. Pizza
5. Histograma
6. Barras com dados de examples/vendas.txt
7. Barras e exportar para grafico.png (sem janela)
```

As cinco primeiras opções exercitam cada tipo de gráfico com dados embutidos no código; a opção 6 usa `load_data` e `free_data`; a opção 7 usa `export_png`. Em todas, o programa verifica o retorno dos construtores (`window.width == 0`, `data.size == 0`) antes de prosseguir, seguindo a convenção de erros da biblioteca. Juntas, a aplicação e os testes utilizam todas as doze funções públicas.

8. Compilação com Makefile

8.1. Alvos

O Makefile implementa os cinco alvos exigidos e alguns adicionais, descritos na Tabela 4.

Tabela 4 – Alvos do Makefile

Alvo	Descrição
`all`	Padrão. Gera as bibliotecas e os dois executáveis da aplicação.
`libs`	Gera `lib/libmatplotc.a` e `lib/libmatplotc.so`.
`app`	Gera `bin/matplotc_static` e `bin/matplotc_dynamic`.
`run`	Compila, se necessário, e executa a aplicação ligada estaticamente.
`test`	Compila cada `tests/*.c` em um executável e executa todos, indicando `APROVADO` ou `FALHA`.
`install`	Instala bibliotecas em `$(PREFIX)/lib` e cabeçalhos públicos em `$(PREFIX)/include/matplotc`.
`clean`	Remove `build/`, `lib/` e `bin/`.
`deps`	Instala ou compila a raylib (`scripts/setup-deps.sh`).
`check-deps`	Verifica se a raylib está disponível.

Fonte: Os autores.

8.2. Estratégia de compilação incremental

Os fontes são descobertos automaticamente com `$(wildcard src/*.c)`, e uma regra de padrão compila cada um em um objeto separado em `build/obj`:

```
CFLAGS := -Wall -Wextra -std=c11 -O2 -Iinclude -Iinclude/
   internal -MMD -MP

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c | $(OBJ_DIR)
        $(CC) $(CFLAGS) -fPIC -c $< -o $@

-include $(DEPS)
```

As opções `-MMD` e `-MP` fazem o GCC gerar, junto a cada objeto, um arquivo `.d` com a lista de cabeçalhos que aquele fonte realmente inclui; a diretiva `-include $(DEPS)` incorpora essas regras ao Makefile. Assim, as dependências entre arquivos não precisam ser escritas à mão e permanecem corretas quando os `#include` mudam. Por exemplo, o arquivo gerado para `pie.o` é:

```
build/obj/pie.o: src/pie.c include/internal/pie.h include/data.h
    \
 include/plane.h include/internal/plotarea.h include/context.h \
 include/internal/colours.h raylib.h include/internal/data_draw.
    h
```

O diretório de saída aparece como pré-requisito *order-only* (depois do `|`): ele precisa existir, mas sua data de modificação não dispara recompilações. Os comportamentos obtidos na prática foram:

- sem alterações, `make` responde `Nothing to be done for 'all'`;
- após alterar apenas `src/pie.c`, somente `pie.o` é recompilado, seguido da regeração das bibliotecas e da religação da aplicação;
- após alterar `include/plane.h`, são recompilados apenas os nove fontes que o incluem direta ou indiretamente (`bar`, `data`, `hist`, `line`, `load`, `matplotc`, `pie`, `plane` e `scatter`); `context.c`, `colours.c` e `plotarea.c` não são tocados.

```
$ make
make: Nothing to be done for 'all'.
$ touch src/pie.c && make
gcc ... -fPIC -c src/pie.c -o build/obj/pie.o
ar rcs lib/libmatplotc.a build/obj/bar.o ... build/obj/scatter.o
gcc -shared -o lib/libmatplotc.so build/obj/bar.o ... build/obj/
   scatter.o ...
gcc ... app/main.c lib/libmatplotc.a -o bin/matplotc_static ...
gcc ... app/main.c -Llib -lmatplotc ... -o bin/matplotc_dynamic
   ...
```

8.3. Instalação

O alvo `install` usa o utilitário `install` para copiar as duas bibliotecas e somente os cabeçalhos públicos. O prefixo padrão é `/usr/local`, mas pode ser alterado para uma instalação sem privilégios administrativos; a variável `DESTDIR` também é respeitada, o que facilita o empacotamento:

```
$ make install PREFIX=$HOME/.local
Instalado em /home/usuario/.local/lib e /home/usuario/.local/
   include/matplotc
```

Ao final, o prefixo contém `lib/libmatplotc.a`, `lib/libmatplotc.so` e `include/matplotc/` com `matplotc.h`, `context.h`, `data.h` e `plane.h`. Os cabeçalhos de `include/internal/` não são instalados.

9. Bibliotecas estática e dinâmica

Os mesmos objetos de `build/obj` são usados para gerar as duas bibliotecas. Por isso todos são compilados com `-fPIC` (código independente de posição), requisito para a biblioteca dinâmica e inofensivo para a estática.

9.1. Biblioteca estática

A biblioteca estática é um arquivo de objetos criado com o `ar`:

```
$(STATIC_LIB): $(OBJS) | $(LIB_DIR)
        ar rcs $@ $^

$(APP_STATIC): $(APP_SRC) $(STATIC_LIB) $(HEADERS) | $(BIN_DIR)
        $(CC) $(CFLAGS) $(LDFLAGS) $(APP_SRC) $(STATIC_LIB) -o
           $@ $(LDLIBS)
```

As opções `r`, `c` e `s` inserem ou substituem os objetos, criam o arquivo se necessário e geram o índice de símbolos. Na ligação, o *linker* copia para o executável os objetos da biblioteca de que a aplicação precisa; o resultado, `bin/matplotc_static`, não depende da `libmatplotc` em tempo de execução.

9.2. Biblioteca dinâmica

A biblioteca dinâmica é gerada com `-shared`, e a aplicação é ligada a ela por nome (`-lmatplotc`):

```
$(SHARED_LIB): $(OBJS) | $(LIB_DIR)
        $(CC) -shared -o $@ $^ $(LDLIBS)

$(APP_SHARED): $(APP_SRC) $(SHARED_LIB) $(HEADERS) | $(BIN_DIR)
        $(CC) $(CFLAGS) $(LDFLAGS) $(APP_SRC) -L$(LIB_DIR) -l$(
           LIB_NAME) \
            -Wl,-rpath,'$$ORIGIN/../lib' $(RUNTIME_RPATH) -o $@
               $(LDLIBS)
```

O `rpath` relativo `$ORIGIN/../lib` instrui o carregador dinâmico a procurar a biblioteca no diretório `lib/` ao lado de `bin/`. O `ldd` confirma que, ao contrário do executável estático, o executável dinâmico efetivamente carrega a `libmatplotc.so` em tempo de execução:

```
$ ldd bin/matplotc_dynamic | grep -E 'matplotc|raylib'
  libmatplotc.so => .../bin/../lib/libmatplotc.so
```

```
  libraylib.so.550 => .../lib/libraylib.so.550
$ ldd bin/matplotc_static | grep -E 'matplotc|raylib'
  libraylib.so.550 => .../lib/libraylib.so.550
```

Em ambos os casos a raylib aparece como dependência dinâmica, porque o ambiente usado possuía apenas a versão compartilhada dela. A ligação estática diz respeito à `libmatplotc`.

9.3. Comparação entre ligação estática e dinâmica

Na ligação estática, o código da biblioteca é copiado para dentro do executável no momento da ligação. Na dinâmica, o executável guarda apenas uma referência à biblioteca, que é carregada e ligada pelo carregador do sistema (`ld.so`) quando o programa inicia. No matplotc, essa diferença é visível no tamanho dos executáveis: 39.784 bytes para `matplotc_static` contra 16.488 bytes para `matplotc_dynamic`. A Tabela 5 resume as diferenças.

Tabela 5 – Comparação entre ligação estática e dinâmica

Aspecto	**Estática (`.a`)**	**Dinâmica (`.so`)**
Momento da ligação	Compilação.	Carregamento do programa.
Tamanho do executável	Maior (contém o código usado).	Menor (contém só referências).
Dependência na execução	Nenhuma da `libmatplotc`.	A `.so` precisa ser encontrada (`rpath`, `LD_LIBRARY_PATH` ou diretórios do sistema).
Atualização da biblioteca	Exige religar a aplicação.	Basta substituir a `.so`, se a interface binária for mantida.
Memória	Cada processo tem sua cópia.	Páginas de código compartilhadas entre processos.
Desempenho	Sem custo de carregamento; chamadas diretas.	Pequeno custo inicial e chamadas indiretas (PLT).
Requisito de compilação	Nenhum especial.	Objetos com `-fPIC` e ligação com `-shared`.

Fonte: Os autores.

10. Testes automatizados

10.1. Estratégia

Para não depender de bibliotecas externas, o grupo escreveu um pequeno *framework* de asserções em `tests/framework.h`, composto pelas macros `MT_BEGIN`, `MT_CHECK`, `MT_CHECK_INT`, `MT_CHECK_FLOAT` (com tolerância), `MT_CHECK_STR` e `MT_END`. Cada falha imprime a expressão, os valores esperado e obtido, o arquivo e a linha:

```
#define MT_CHECK_FLOAT(expected, actual, epsilon) \
  do { \
    double mt_e = (double)(expected); \
    double mt_a = (double)(actual); \
    total++; \
    if (fabs(mt_e - mt_a) > (double)(epsilon)) { \
      failed++; \
      printf("␣␣FALHOU␣␣%s␣==␣%s␣␣(esperado␣%g,␣obtido␣%g)␣␣" \
             "(%s:%d)\n", #expected, #actual, mt_e, mt_a, \
             __FILE__, __LINE__); \
    } \
  } while (0)
```

Cada arquivo `tests/*.c` é compilado contra a biblioteca estática em um executável próprio, em `build/tests/`. O alvo `make test` executa todos, acumula as falhas e encerra com código diferente de zero se algum teste falhar, o que o torna utilizável em integração contínua.

Como as funções que desenham dependem de um contexto gráfico, os testes foram organizados para que a maior parte rode sem tela: os construtores, o carregamento de arquivos e todos os caminhos de erro de `show` e `export_png` são verificados sem abrir janela. O teste de exportação real só é executado quando há um *display* disponível (variável `DISPLAY`), e é ignorado em ambientes sem tela.

10.2. Cobertura

A Tabela 6 relaciona os arquivos de teste ao que eles cobrem. Os arquivos de dados usados ficam em `tests/fixtures/`: um arquivo válido com comentários e separadores mistos, um com linhas malformadas, um com 100 pontos (para forçar o crescimento dos vetores além da capacidade inicial) e um sem nenhum ponto válido.

Tabela 6 – Arquivos de teste e cobertura

Arquivo	Cobre	Asserções
`test_main.c`	`create_context` e `create_plane` (casos válidos e todas as entradas inválidas), `load_data` básico e caminhos de erro de `show`, `plot_bar`, `plot_histogram` e `export_png`.	49
`test_data.c`	`create_data` (válido e sete combinações inválidas), `load_data` (arquivo válido, linhas malformadas, crescimento de capacidade, arquivo inexistente, arquivo vazio, parâmetros nulos) e `free_data` (idempotência, `NULL` e dados emprestados).	51
`test_export.c`	`export_png`: falhas sem janela e, com *display*, geração real de um PNG não vazio a partir de `plot_line`.	5 + 2

Fonte: Os autores.

10.3. Resultados

Todos os testes passaram. A saída resumida de `make test` é:

```
==> build/tests/test_data
>>> data: create_data
    PASSOU: 15/15 assercoes
>>> data: load_data (arquivo valido)
    PASSOU: 13/13 assercoes
>>> data: load_data (linhas invalidas sao ignoradas)
    PASSOU: 9/9 assercoes
>>> data: load_data (cresce a capacidade)
    PASSOU: 4/4 assercoes
>>> data: load_data/free_data (erros)
    PASSOU: 10/10 assercoes
==> build/tests/test_export
>>> export_png: falhas cedo (sem janela)
    (sem DISPLAY: export real ignorado)
    PASSOU: 5/5 assercoes
==> build/tests/test_main
>>> context and plane constructors
    PASSOU: 49/49 assercoes
RESULTADO: APROVADO
```

Executado com um servidor X virtual (`xvfb-run`), o `test_export` também gera o PNG e passa nas duas asserções adicionais. As mensagens de erro impressas pela biblioteca em `stderr` durante os testes são esperadas: elas resultam justamente dos casos inválidos testados.

Como limitação, as funções de desenho (`plot_line`, `plot_scatter`, `plot_pie`) só têm seus caminhos de rejeição verificados sem tela; a verificação do resultado visual depende do teste de exportação e da inspeção das imagens geradas.

10.4. Teste que falhou inicialmente e posterior correção

O histórico atual do repositório não registra explicitamente um teste que falhou antes de uma correção. Uma demonstração adequada, baseada em um defeito real do código, é a seguinte: as funções de linha, dispersão e legenda acessam `COLOURS[j]` sem aplicar o resto da divisão por `COLOURS_COUNT`, de modo que um gráfico com mais de oito séries lê fora dos limites da paleta (os módulos de barras e histograma já usam `j % COLOURS_COUNT`). Extraindo a escolha de cor para uma função interna, por exemplo `Color series_colour(int index)`, escreve-se primeiro o teste:

```
1 MT_CHECK(colour_equal(series_colour(8), series_colour(0)));
2 MT_CHECK(colour_equal(series_colour(9), series_colour(1)));
```

que falha enquanto a função devolve `COLOURS[index]`, e passa após a correção para `COLOURS[index % COLOURS_COUNT]`. A sequência deve ser registrada em dois *commits* (teste e correção) e demonstrada com `make test` antes e depois.

11. Versionamento com Git

O projeto foi desenvolvido no GitHub com o fluxo de *feature branches*: cada funcionalidade, correção ou refatoração foi desenvolvida em uma *branch* própria e integrada à `main` por um *pull request*, gerando um *commit* de *merge*. Contribuições de integrantes que trabalharam em *forks* também foram integradas dessa forma. As mensagens de *commit* seguem a convenção *Conventional Commits*, com prefixos que indicam a natureza da mudança: `feat`, `fix`, `refactor`, `tests`, `docs`, `chore` e `style`.

O histórico mostra uma evolução incremental e coerente: primeiro os gráficos de dispersão e de linhas, depois o Makefile e o *script* de dependências, a base de testes, a modularização, os construtores, a migração para `float`, o histograma, a separação entre registro e desenho, o carregamento de arquivos com a exportação PNG e, por último, a separação dos cabeçalhos internos e a cobertura completa de testes.

11.1. Branches e merges

A Tabela 7 lista as principais *branches* integradas por *pull request*.

Tabela 7 – Principais branches integradas à `main`

PR	Branch	Conteúdo
#2	`feat/add-makefile`	Makefile, `setup-deps.sh`, `.gitignore`.
#3	`modules`	Primeira divisão do código em módulos.
#4	`tests/add-testing-foundation`	*Framework* de testes.
#5	`feat/struct-constructors`	Construtores das estruturas.
#7	*fork* `felipe-olima/main`	Histograma, aplicação de exemplo e testes de contexto e plano.
#9	`fix/change-discrete-to-continuos-in-hist`	Histograma com variável contínua.
#10	`fix/remove-axis-from-pie`	Pizza sem eixos e com legenda por fatia.
#12	`refactor/reorganize-modules`	Separação entre registro e desenho.
#13	`feat/load-data-from-file`	`load_data`, `free_data` e `export_png`.
#14	`refactor/clear-and-organize-headers`	Diretório `include/internal/`.
#15	`docs/complete-readme-and-tests`	README e cobertura completa de testes.

Fonte: Os autores.

A *branch* `feat/load-data-from-file` é um bom exemplo do fluxo: ela acrescentou o arquivo `src/load.c`, as funções `load_data` e `free_data`, o campo `owns_memory`, a função `export_png`, o arquivo de exemplo e os testes correspondentes, e foi integrada pelo *merge* `efc2ab5`. Alguns *merges* exigiram resolução de conflitos, registrada nos *commits* `e322012` e `e406d63`. A estrutura pode ser visualizada com:

```
git log --oneline --graph --all
git branch -a
```

12. Dificuldades encontradas e decisões de projeto

As principais dificuldades e as decisões que elas motivaram foram:

- **Testar código gráfico.** Na versão inicial, cada `plot_*` abria uma janela e bloqueava, o que impedia testes automáticos. A solução foi separar o registro do gráfico da renderização e criar a exportação por janela oculta, permitindo testar quase toda a API sem tela.
- **Fronteira entre API pública e implementação.** Inicialmente, todas as funções de desenho estavam nos cabeçalhos públicos. A criação de `include/internal/` reduziu a API ao que o usuário realmente precisa e permitiu evoluir os módulos de desenho sem afetar a aplicação.
- **Tipos numéricos.** O uso de `int` restringia os dados a valores inteiros e causava divisões inteiras nas escalas. A migração para `float` resolveu os dois problemas.
- **Semântica dos gráficos.** O histograma tratava os valores como discretos e a pizza desenhava eixos sem sentido; ambos foram corrigidos em *branches* próprias.
- **Dependência externa.** A raylib nem sempre está disponível nos repositórios das distribuições. Por isso, o grupo criou o *script* `setup-deps.sh`, que a compila

a partir do código-fonte quando necessário, e a variável `RAYLIB_PREFIX` no Makefile.
- **Posse da memória.** Permitir dados vindos tanto de vetores do usuário quanto de arquivos exigiu uma regra explícita de posse, implementada pelo campo `owns_memory`.

13. Resultados obtidos e possíveis melhorias

O matplotc atende aos requisitos funcionais do trabalho: possui doze funções públicas distribuídas em quatro módulos com responsabilidades próprias, interface pública definida nos cabeçalhos de `include/`, bibliotecas estática e dinâmica efetivamente utilizadas pela aplicação, Makefile com compilação incremental baseada em dependências geradas pelo compilador, testes automatizados executados por `make test` e histórico Git incremental com *branches* e *merges*. O resultado visual pode ser conferido nas Figuras 1 a 5.

A análise do código e a compilação em um ambiente limpo revelaram as seguintes limitações, que indicam os próximos passos:

- **Pizza com posição fixa.** O gráfico de pizza é sempre desenhado com centro em $(400, 400)$ e raio de 150 pixels, ignorando o tamanho da janela e a `PlotArea`. O centro e o raio deveriam ser derivados da área de desenho.
- **Paleta limitada.** Os gráficos de linha, dispersão e a legenda indexam `COLOURS[j]` sem o resto da divisão, acessando memória inválida com mais de oito séries (Seção 10.4).
- **Um gráfico por vez.** O estado global permite um único gráfico registrado e exige que os dados sobrevivam até a renderização. Uma evolução natural é uma estrutura de figura explícita, com suporte a vários gráficos (*subplots*).
- **Rótulos e legenda.** Os rótulos dos eixos são sempre lidos da primeira série, e o campo `plane` de `Data` é armazenado, mas não é usado pelo desenho.
- **Símbolos internos exportados.** A `libmatplotc.so` exporta também as funções internas (`draw_plane`, `create_bar` etc.). Compilar com `-fvisibility=hidden` e marcar apenas a API como visível restringiria a interface binária à API pública.
- **Makefile com raylib fora do sistema.** A regra da `.so` não repassa `$(LDFLAGS)`, e a regra dos testes não inclui o `rpath` da raylib. Com `RAYLIB_PREFIX`, por isso, foi necessário exportar `PKG_CONFIG_PATH` e `LD_LIBRARY_PATH` manualmente.
- ***Script* de dependências.** Em uma distribuição sem o pacote `libraylib-dev`, o *script* encerrou com erro de variável não definida (`cc` e `tmp`) por causa do `set -u`.
- **Ordem de `export_png` e `show`.** A opção de janela oculta só vale na primeira janela criada pelo processo, então `export_png` deve ser chamada antes de `show`.
- **Ajustes menores.** A aplicação ignora o retorno de `scanf` (aviso do compilador), e cálculos como as frequências do histograma e a regressão são refeitos a cada quadro.

Entre as funcionalidades extras sugeridas no enunciado, as mais naturais para o projeto são a integração contínua com GitHub Actions (os testes já foram pensados para

rodar sem tela, e o teste de exportação pode usar `xvfb-run`), a geração de documentação com Doxygen a partir dos comentários dos cabeçalhos e a medição de cobertura com `gcov`.

14. Considerações finais

O desenvolvimento do matplotc mostrou que é possível oferecer, em C, uma experiência de plotagem próxima à de bibliotecas de linguagens de mais alto nível, com uma API pequena e previsível. As decisões que mais contribuíram para isso foram a separação entre registrar e renderizar o gráfico, a divisão entre interface pública e interna e a regra explícita de posse da memória. Essas mesmas decisões facilitaram os testes automatizados e a geração das bibliotecas estática e dinâmica. As limitações listadas, por sua vez, formam um roteiro concreto para as próximas versões.